Technical Documentation

Enterprise Document Q&A using Gemini + Agentic RAG

---

1. Introduction

This project is a **Generative AI-based document intelligence system** that enables users to query enterprise documents using natural language.

The system leverages:

- **Google Gemini (LLMs + Embeddings)**
- **Retrieval-Augmented Generation (RAG)**
- **Agentic AI workflow** for structured reasoning

---

2. System Objectives

- Enable semantic search over enterprise documents
- Provide accurate, context-aware responses
- Reduce hallucinations using grounded retrieval
- Demonstrate end-to-end Agentic AI pipeline

---

3. High-Level Architecture

Workflow

```
User → Upload Documents → Chunking → Embeddings → Vector Store
     → Query → Planner → Retriever → Reasoner → Verifier → Answer
```

Components

1. Document Ingestion
2. Text Chunking
3. Embedding Generation
4. Vector Search
5. Agentic Pipeline
6. UI Layer (Streamlit)

---

4. Module Breakdown

4.1 File Parsing Module

Handles multi-format document ingestion:

- PDF → Extracts page-wise text
- TXT → Reads raw text
- CSV → Converts structured data to text
- Excel → Reads sheets and converts to text

Key Functions:

- `read_pdf()`
- `read_txt()`
- `read_csv()`
- `read_excel()`
- `parse_uploaded_file()`

4.2 Text Chunking Module

Splits documents into smaller chunks for better retrieval.

Features:

- Fixed chunk size
- Overlapping windows
- Smart sentence boundary detection

Key Function:

- `chunk_text()`

4.3 Embedding & Vector Search

Embedding

- Uses Gemini embedding model
- Converts text into numerical vectors

Vector Search

- Cosine similarity for semantic matching
- Returns top-k relevant chunks

Key Functions:

- `embed_texts()`
- `cosine_similarity()`
- `search_similar_chunks()`

4.4 Agentic Workflow

1. Planner Agent

- Determines query intent
- Defines retrieval strategy

2. Retriever Agent

- Performs semantic search
- Returns relevant document chunks

3. Reasoning Agent

- Generates grounded answers
- Uses retrieved context

4. Verifier Agent

- Validates output
- Removes hallucinations
- Ensures citation correctness

---

5. Data Flow

Step-by-Step

1. User uploads documents
2. Documents are parsed and converted to text
3. Text is chunked into smaller segments
4. Each chunk is embedded using Gemini
5. Embeddings stored in memory
6. User submits query
7. Query is embedded
8. Similar chunks are retrieved
9. LLM generates answer using context
10. Verifier refines final output

---

6. Prompt Engineering Strategy

Reasoning Prompt

- Strict grounding rules
- Citation enforcement
- No hallucination policy

Verifier Prompt

- Checks unsupported claims
- Adds uncertainty if needed
- Improves clarity and correctness

---

7. Error Handling & Guardrails

- Input validation for files
- API key validation via `.env`
- Empty query handling
- Context length control
- Verifier agent to reduce hallucinations

---

8. Limitations

- In-memory vector storage (no persistence)
- Limited scalability for large datasets
- Dependent on LLM API availability
- No authentication layer

---

9. Future Enhancements

- Persistent vector DB (Chroma, FAISS)
- Multi-agent orchestration frameworks (LangGraph)
- Authentication (OTP, OAuth)
- Cloud deployment (AWS, GCP)
- Evaluation metrics (RAGAS)

---

10. Deployment Considerations

- Use Docker for portability
- Secure API keys using environment variables

- Monitor API usage and cost
- Optimize chunk size and retrieval parameters

---

11. Security Considerations

- Avoid storing sensitive documents
- Use secure API key management
- Implement user authentication for production

---

12. Conclusion

This project demonstrates a complete **end-to-end Generative AI system** combining:

- LLMs
- RAG
- Agentic AI

It serves as a strong foundation for building enterprise-grade AI assistants.

---

13. Glossary

- **RAG:** Retrieval-Augmented Generation
- **LLM:** Large Language Model
- **Embedding:** Numerical representation of text
- **Chunking:** Splitting text into smaller parts
- **Agentic AI:** Multi-step reasoning using autonomous agents

---